

Demo runbook

Visitor Counting Console

Contents

Demo runbook	1
1. Fifteen minutes before	2
2. Starting it	2
3. What to show, in order	3
4. If something goes wrong	5
5. Afterwards	6

Demo runbook

Running the visitor console for a demo, on any of the three machines it is set up on. Everything from §3 onwards is the same everywhere; only starting and stopping differ.

	machine	how it runs
Ubuntu	the demo box with an NVIDIA card, repo in e.g. <code>~/Documents/cv-demo-humanmonitor</code>	a systemd service, <code>visitor-console</code>
Windows	a demo PC with an NVIDIA card, repo in e.g. <code>C:\cv-demo-humanmonitor</code>	<code>run_web.bat</code> in a terminal window
macOS	Apple Silicon laptop, the walkthrough machine	<code>./run_web.sh</code> in Terminal

The console is on **port 8780** on all three.

Two ways to feed it, on all three:

	when
live — the RTSP cameras	showing the real installation
video — the five saved clips	showing the features

Demonstrate on the clips, not on live cameras. A camera pointed at a quiet corridor produces an empty frame, and an empty frame demonstrates nothing: the counters sit still and there is no way to show that any of it works. The clips are busy on purpose, so the numbers move while you are talking.

macOS is the weakest of the three. No CUDA, so the pipeline runs on the Apple GPU: three 720p files give roughly 5-10 pipeline fps. Fine for a walkthrough on a laptop, not for standing in front of a customer with live cameras. Use the Ubuntu box for that.

1. Fifteen minutes before

The same three checks everywhere — only the path to the interpreter differs:

	interpreter
Ubuntu / macOS	<code>.venv/bin/python</code>
Windows	<code>.venv\Scripts\python.exe</code>

Ubuntu / macOS

```
cd ~/Documents/cv-demo-humanmonitor      # macOS: wherever the clone is
git fetch origin
git status -sb                            # want: "## main...origin/main" and nothing else
ls -lh sample_data/videos/               # five clips, ~110 MB total
```

Windows

```
cd /d C:\cv-demo-humanmonitor
git fetch origin
git status -sb
dir sample_data\videos
```

`git status -sb` answers both questions at once. [behind N] on the first line means this machine is missing N commits — `git pull`. Any lines beneath it are local changes; if they are not yours, see the reset below. (This used to name a specific commit to check against, which went stale the moment anything was pushed.)

If the clips are missing, or are ~130 bytes:

```
.venv/bin/python download_demo_videos.py    # Ubuntu / macOS
.venv\Scripts\python.exe download_demo_videos.py  :: Windows
```

They are git-ignored on purpose, so downloading them never dirties the tree.

If `git status` is not empty and the changes are not yours — most likely an interrupted `git pull` — put the tree back in step:

```
git fetch origin && git reset --hard origin/main
```

That only touches tracked source. Your cameras, trigger lines, counting areas, hours, history and recordings all live in git-ignored files and are untouched.

2. Starting it

Ubuntu — the demo box

Live cameras run as the service:

```
sudo systemctl restart visitor-console
systemctl is-active visitor-console      # expect: active
```

For the saved clips, stop the service first — both want port 8780:

```
sudo systemctl stop visitor-console
./run_web.sh --mode video --capacity 150
```

Leave that terminal open; `Ctrl+C` stops it. Afterwards: `sudo systemctl start visitor-console`. To run the clips *alongside* the live service, give them a second port and open `http://localhost:8781`:

```
./run_web.sh --mode video --port 8781
```

Windows

No service — it runs in a terminal window that stays open:

```
run_web.bat --mode video --capacity 150
run_web.bat --mode live --capacity 150
```

`Ctrl+C` in that window stops it; closing the window also does. On the first start Windows Defender Firewall asks whether to allow Python on private networks — **Allow** if another PC on the LAN should open the console.

If the PC is set up to start the console at logon (Task Scheduler task `VisitorConsole`, §4.5 of `DEPLOYMENT.md`), that instance already holds 8780; stop it first or use `--port 8781`:

```
Stop-ScheduledTask -TaskName VisitorConsole
```

macOS

```
./run_web.sh --mode video --capacity 150
./run_web.sh --mode live --capacity 150
```

`Ctrl+C` stops it. If `pipeline fps` is low, `--device cpu` is sometimes faster than the Apple GPU for the classifier, and `--no-demographics` turns the male/female and adult/child work off entirely — worth knowing, but it also turns off half of what the demo is about, so prefer the clips at a smaller `--stream-width 640`.

Then, in the browser, on all three

Open <http://localhost:8780> and press **Ctrl+Shift+R** (**Cmd+Shift+R** on macOS). The panel changes often and the browser will otherwise serve a cached stylesheet — if the layout does not match this runbook, the refresh did not take.

The **building day set-up** screen appears first: confirm the date, opening and closing times, capacity and starting occupancy, then **Start monitoring**. It asks once per day.

3. What to show, in order

1. **The panel in one view** — VISITORS TODAY, CURRENT OCCUPANCY, CAMERAS DEPLOYED side by side. Every camera gets its own row in the third column; past five the list scrolls on its own rather than pushing anything else off the panel.
2. **The two figures are different things** — visitors today counts everyone who entered; current occupancy is who is inside now. Each box sums to the two tiles beneath it.
3. **The icons** — male/female and adult/child are told apart by shape, so they still read in greyscale or from across a room.
4. **Per camera** — click a card under CAMERAS DEPLOYED and the whole panel follows that entrance; click it again for the whole site.
5. **On the frames** — the same numbers are drawn into each camera image, so a recorded clip carries them.

6. **Swap a feed** — hover a camera panel and press **v**; **Shift+V** swaps all of them. Also the **> live / > video** button on each panel.
7. **Trigger lines** — *calibrate*, drag an end point, *save*. Two things on the overlay are worth naming before anyone asks: the faint dashed box is where the *picture* ends (the panel is wider than the frame, so the image is letterboxed inside it), and the faded dashed extension past each end of the line is the **8 % of its length that still counts** — drawn so the reach is not a surprise in the numbers. Placing a line within 4 % of the frame edge warns: people can cross it before the tracker has established them.
8. **Drawn areas** — *calibrate*, then *draw area*, pick what the area is for from the dropdown next to it, then **drag a box** across the picture (or click two opposite corners). Drag when the box needs to reach an edge: the button strip sits on top of the picture, and only the *start* of a drag has to miss it. The box is dashed on the frame in the colour of its role:

role	what it does	line may sit outside?
count only inside (orange)	only people inside are tracked at all — outside the box is dimmed	no — a warning appears
ignore inside (red)	people inside are skipped; use it on a window, a mirror or a poster	yes
classify only inside (blue)	everyone is counted, but only those inside are sexed and aged	yes

classify only inside is the one to demonstrate for accuracy: put it where people are large and well lit, and the crop budget stops being spent on distant figures. Tracks outside it still count, they just read **??**. *clear area* puts the whole frame back. A box has to be at least **5 % of the frame** on each side; smaller than that is refused, since it is more likely a stray click than an area anyone meant to draw.

While a box is being drawn the buttons that do not belong to that flow — *flip IN*, *clear*, *delete line*, the feed swap and *record* — grey out on purpose. They sit on the picture, and a corner placed near the bottom of the frame used to hit one of them instead. Areas save to `config/lines.json` beside the trigger lines, so anything set up in rehearsal is still there after a restart.

9. **What the area holds right now** — a counting area reports its own population without waiting for anyone to cross the line. The box carries it on the frame (**COUNTING AREA / 3 HERE 1M 2F 3A 0C**) and the panel repeats it as **in area now**. A trailing **?** is people the classifier has not decided yet, so the splits always add up to the total. This is a *presence* figure, not a crossing figure: it goes up and down as people walk through shot, and it is the one to point at when the demo starts mid-scene and the crossing counters are still at zero. With no area drawn the same line reads **in view now** and covers the whole frame.
10. **Capacity alarm** — *site*, set capacity below the current count. The panel banner, the alert row, the camera ribbon and that camera's card all turn red.
11. **Area capacity** — [settings] on a camera, set an area capacity of 3. That camera alarms on its own. Note this is a crossing figure and is not the same as *in area now* above.
12. **Record an event** — ** record* saves 10 s before and 20 s after with a note, into `output/events/` and the history.

13. **History** — [expand] expands the console: alerts, cameras, crossings, hourly and closing totals, all written to `output/history.csv`.

4. If something goes wrong

symptom	cause and fix
address already in use	something already holds 8780. Ubuntu: the service — <code>systemctl is-active visitor-console</code> , then use it or <code>sudo systemctl stop visitor-console</code> . Windows: the logon task — <code>Stop-ScheduledTask -TaskName VisitorConsole</code> , or <code>netstat -ano findstr :8780</code> for the PID. macOS: <code>lsof -ti:8780</code> . Or just add <code>--port 8781</code>
layout looks like an older version	cached CSS — Ctrl+Shift+R , or Cmd+Shift+R on macOS
a camera shows OFFLINE	the stream is unreachable. The card hides its counts on purpose: the numbers it had were frozen at the moment the link dropped
connect hangs when adding a camera	should not happen since 4417398 — opens are capped at 5 s. If it does, the box is on an older commit
everyone on one camera reads ??	a <i>classify only inside</i> area that nobody walks through, or one covering only distant figures — move it, enlarge it, or <i>clear area</i>
a camera counts nothing after an area was drawn	the area is <i>count only inside</i> and the trigger line is not inside it — people are first seen already at the line. Either enlarge the area to cover the whole line with room on both sides, or switch the role to <i>classify only inside</i> . Saving it warns about this
counts negative, or 0 inside with a note	the clips start mid-scene, so people leave through lines they were never seen entering. reset counters in the footer, or set a starting occupancy on the set-up screen
v does nothing, toast says no paired feed	that slot has no counterpart. <code>config/sources.json</code> needs <code>live</code> and <code>video</code> lists of the same length — slot <i>i</i> pairs with the other list's entry <i>i</i>
a camera added on the page will not connect, and the password has symbols in it	@ : \$ % & and spaces are encoded for you - paste them as they are. / ? # must be typed as %2F %3F %23. If you passed the URL on a command line instead, single-quote it: \$\$ unquoted is the shell's process id
cameras read Camera 1 / 2 / 3	no display names set. [settings] on a panel, type a name, connect. It persists
git pull refused	see §1

symptom	cause and fix
pipeline fps in low single figures	expected on macOS (no CUDA) — see the note at the top. On Ubuntu or Windows it means the GPU is not being used: check the log's first lines for <code>CUDA available: True</code>
Windows: SmartScreen or antivirus blocks the launcher	allow <code>.venv\Scripts\python.exe</code> ; keep the folder out of a OneDrive-synced path
macOS: OpenCV windows never appear	<code>run_demo.sh</code> needs a logged-in desktop session, not SSH. The browser console is the headless option

Logs. Ubuntu service: `journalctl -u visitor-console -f`. Everywhere else: the terminal window it is running in.

Health check, the same on all three:

```
curl -s http://localhost:8780/api/state | head -c 300
```

```
curl.exe -s http://localhost:8780/api/state           # Windows PowerShell
```

Look for "error": null, each camera "status": "LIVE" or "FILE", and a pipeline_fps above zero.

5. Afterwards

Ubuntu

```
sudo systemctl start visitor-console           # if you stopped it for the clips
```

Windows — close the terminal window, or `Start-ScheduledTask -TaskName VisitorConsole` if the PC normally runs it at logon.

macOS — `Ctrl+C` in Terminal.

On all three: the day's numbers are in `output/history.csv`; recorded clips and snapshots in `output/events/`. Neither is in git — copy them off if they are needed. Trigger lines and counting areas stay in `config/lines.json`, so the next demo starts calibrated.

Counters reset at the opening time set on the set-up screen.